

Reinforcement Learning in Robotics and Sim2Real Transfer

Alessio Gerardo Demattia
s332683

Abstract—In recent years, Reinforcement Learning has taken on an increasingly central role in robotic control, thanks to its ability to achieve adaptive behaviour in complex and dynamic environments, overcoming the limitations of direct programming. However, a significant obstacle that persists in practical application is the so-called *sim-to-real gap*, i.e., the difficulty of transferring policies learned in simulation to real robotic platforms. In this sense, the project initially focuses on training a single-legged robot to walk using classical algorithms (REINFORCE and Actor-Critic) and more advanced methods, such as Soft Actor-Critic (SAC). Therefore, it addresses the problem of transferring the policies learned through domain randomization on the dynamic parameters of the simulation. To improve the results of manual randomization tuning, the SimOpt framework is implemented, which iteratively adapts the distribution to match real behaviour using a few real-world roll-outs.

I. INTRODUCTION

Reinforcement Learning (RL) is a field of study at the intersection of Machine Learning and control theory. It aims to teaching an agent how to behave in a dynamical and interactive environment in order to maximize some sort of reward. The RL paradigm does not search a function able to generalize a dataset input-output, like supervised learning, neither it tries to spot unseen patterns, like unsupervised learning; instead, RL attempts to solve a multi-stage optimization problem through trial-and-error approach algorithms.

In the last decade, RL has demonstrated its strength in many domains, especially in discrete environments, in games (e.g., Atari [1], Chess [2], Hex [3]), and a lot of other problems as autonomous driving, planning, non-linear dynamical system control seem to benefit from this approach.

A keen-witted application of RL is robotic. Training robots to move is notoriously difficult. The classical hard-coded and imitation learning approach often turn out to be impractical in complex, dynamic and unpredictable environments [4], [5]. In contrast RL appears to be an appropriate approach: robot acts as an agent that iteratively improve its policy based on received rewards. In this context, the human programmer does not have to completely understand the dynamics of the environment or take into account all possible scenarios. The human only defines a reward signal that quantifies

task achievement, guiding the agent toward the desired behaviour.

Significant results have been achieved in areas such as dexterity [6]. Although RL theoretically offers an encouraging solution, robotics remain inherently challenging. Numerous issues should be addressed, including high degrees of freedom (the curse of dimensionality), continuous state and action spaces, and noisy observations.

A key challenge lies in simulation. Modern deep RL algorithms require large samples, making it often unfeasible to collect sufficient rollouts from real-world robots due to safety constraints, as well as high costs in terms of energy, time, and money. Consequently, a common strategy is to first train the policy within a simulated environment using a physics engine, and then transfer the learned policy to the real-world system. Simulation is also scalable, customizable, and parallelisable, enabling cheaper and faster training for deep reinforcement learning. However, the simulation is a model based on parameters, such as damping, friction and masses. Tuning parameters to match real-world behaviour is a non-trivial task, in part due to the effort required for measurement and identification. Parameters may be incorrect, unmodelled physical effects may be present, and physical simulators often have inherent limitations (e.g., rigid-body assumptions or discretization of time).

This difficulty in transferring a policy effectively from simulation to the real world is known as the *Sim-to-Real gap*. To mitigate this reality gap, the prevailing approach is domain randomization [7], [8], which has demonstrated significant success in producing robust policies.

The aim of this work is to present both classical and state-of-the-art algorithms, and to address the challenge of policy transfer using a simulated toy target environment designed to account for unmodeled effects. All experiments are conducted with the MuJoCo Hopper environment. The code is available at the GitHub repository.

II. METHODOLOGY

A. Markov Decision Process

The core components of Reinforcement Learning are the *Agent* and the *Environment*. The agent observes the state of the environment, selects an action according to its policy, and the environment transitions to a new state based on the previous state and the chosen action while simultaneously providing the agent with a scalar reward. The agent's objective is to maximize its cumulative reward over a finite or infinite horizon.

Usually, for mathematical convenience, Markov property is assumed, i.e., future states not depend on the previous history if the present state is known. In the Markov Decision Process (MDP) framework [9], we define:

- $\mathcal{S}$: the set of states, representing all possible configurations of the environment. Observations available to the agent may be partial or noisy and therefore may not coincide with the true underlying state.
- $\mathcal{A}$: the set of actions, representing all actions available to the agent. The set of feasible actions depends on the current state s , denoted by $\mathcal{A}(s) \subseteq \mathcal{A}$.
- $p(s' | s, a)$: the state transition probability, specifying the probability of reaching state s' given the current state s and action a .
- $r(s, a)$: the reward function, mapping each state-action pair to a scalar reward. The reward may also be stochastic or depend on s' denoted as $r(s, a, s')$.
- π : the policy, a rule that specifies which action a to take in state s . In the deterministic case, we write $\pi(s)$ for the selected action. In the stochastic case, we write $\pi(a | s)$ for the probability of selecting action a in state s .

Both the state and action spaces may be either discrete or continuous. A *trajectory* is a sequence of states and actions:

$$\tau = \{s_0, a_0, s_1, a_1, \dots, a_{t-1}, s_t\}.$$

Given an initial state distribution $p_0(s)$ and a policy π , the probability of a trajectory under interaction is:

$$p(\tau) = p_0(s_0) \prod_{t=0}^{T-1} \pi(a_t | s_t) p(s_{t+1} | s_t, a_t).$$

Formally, the objective of RL is to find a policy π that maximizes the expected total reward:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right],$$

where $\gamma \in [0, 1]$ is the discount factor, which down-weights future rewards. It is used both for modeling

purposes (e.g., in financial tasks) and to ensure that the infinite sum converges.

An important concept for RL algorithms is the notion of value function, which evaluates how good a the current situation is in terms of the expected future reward when following a policy π . More specifically, the on-policy state value function is defined as:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right],$$

and the on-policy action-state value function is defined as:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right].$$

B. REINFORCE

There are numerous reinforcement learning algorithms, which differ depending on the horizon, the size of the state and action spaces, the type of environment, and the agent's capabilities. The two algorithms presented here are both model-free, on-policy, batch methods. Being model-free, the agent does not have a model of the environment (i.e., it does not know how states transition) and learns exclusively from rewards. Being on-policy, the agent improves using only the experience generated by its current policy, and data from other policies cannot be directly used. Finally, as batch algorithms, the policy is updated only after collecting a set of rollouts.

The foundational algorithm is **REINFORCE** [3], which can be understood as a gradient ascent method. Consider a policy parametrized by θ , for example via a neural network. The objective is to maximize the expected return of the parametrized policy:

$$\theta^* = \arg \max_{\theta} J(\pi_\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right],$$

where $J(\pi_\theta)$ is assumed differentiable with respect to θ . The policy parameters can then be optimized iteratively using gradient ascent:

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\pi_\theta),$$

where α is the learning rate.

Through standard derivation, the gradient of the expected return can be written as:

$$\nabla_{\theta} J(\pi_\theta) = \mathbb{E}_\pi \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_\theta(a_t | s_t) A_t \right],$$

where $A_t = \sum_{k=0}^T \gamma^k r_k$ is the cumulative reward over all rollout. The algorithm collects a sufficient number of rollouts and then estimates the gradient using Monte

Carlo sampling to update the neural network policy parametrized by θ .

Monte Carlo sampling provides an unbiased estimate of the policy gradient, but it can suffer from high variance, which may destabilize convergence.

To address this, several variance reduction techniques are employed, based on the EGLP Lemma [3]. One approach is to use the *reward-to-go* instead of the total return, considering only the cumulative reward from time t onward. Another technique introduces a baseline function $b(s_t)$, which depends on the current state and approximates the expected return from that state. Subtracting this baseline from the return reduces variance while keeping the gradient estimator unbiased.

C. Actor Critic

Actor-Critic methods are a family of on-policy, model-free, policy gradient algorithms that train a critic, i.e., a function that estimates either the state-value function or the action-state value function at each step. Critic is often implemented using a neural network. The estimates from critic are used to compute a better advantages A_t from the policy gradient formula. The policy function is called the actor and is trained using policy gradient ascent. The advantages A_t can be computed in several ways, and some common approaches include:

- **REINFORCE**: previous algorithm can be seen as the simplest form of Actor-Critic, using the reward-to-go. The advantages are unbiased but have high variance:

$$A_t = \sum_{k=t}^T \gamma^{k-t} r_k.$$

- **Baseline state-value function**: the advantage is computed by subtracting a learned state-value baseline. The advantages remain unbiased but still exhibit high variance:

$$A_t = \sum_{k=t}^T \gamma^{k-t} r_k - V(s_t),$$

where $V(s_t)$ is estimated by the neural network critic.

- **TD(1) Temporal Difference**: the advantage is computed using one-step temporal differences. These advantages are unbiased and have lower variance:

$$A_t = r_t + \gamma V(s_{t+1}) - V(s_t),$$

where r_t is the observed reward and γ is the discount factor.

The actor-critic algorithm described here is implemented in batch form, but it can also be realized in an online version.

Algorithm 1 Batch Actor-Critic Algorithm

- 1: Initialize actor parameters θ and critic parameters ϕ
- 2: **for** each iteration **do**
- 3: Collect a batch $D = \{\tau_i\}_{i=1}^B$ of B trajectories by running policy $\pi_\theta(a|s)$
- 4: For each trajectory $\tau_i = \{(s_{i,t}, a_{i,t}, r_{i,t})\}_{t=0}^T$ compute Monte Carlo returns

$$y_{i,t} = \sum_{t'=t}^T \gamma^{t'-t} r(s_{i,t'}, a_{i,t'})$$

- 5: Update critic parameters ϕ by minimizing

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i,t} \left\| \hat{V}_\phi^\pi(s_{i,t}) - y_{i,t} \right\|^2$$

- 6: Compute advantage estimates

$$\hat{A}^\pi(s_{i,t}, a_{i,t}) = r(s_{i,t}, a_{i,t}) + \hat{V}_\phi^\pi(s_{i,t+1}) - \hat{V}_\phi^\pi(s_{i,t})$$

- 7: Estimate policy gradient

$$\nabla_{\theta} J(\theta) \approx \frac{1}{B} \sum_{i,t} \nabla_{\theta} \log \pi_\theta(a_{i,t}|s_{i,t}) \hat{A}^\pi(s_{i,t}, a_{i,t})$$

- 8: Update actor parameters

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

- 9: **end for**
-

D. SAC

The **Soft Actor-Critic (SAC)** algorithm [10] is a state-of-the-art algorithm designed for continuous state and action spaces. It is a model-free method, it is online, as the policy is updated at every step, and off-policy, as it can exploit data generated by different policies. SAC is a continuous state and action algorithm. SAC is recognized for its sample efficiency and for its stability with respect to hyperparameter selection and seed.

The SAC framework is based on *Entropy-Regularized Reinforcement Learning* [3] (from which derives “Soft”). Unlike classical RL, which seeks to maximize the expected cumulative reward, SAC optimizes the following objective:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right],$$

where $\mathcal{H}(\pi(\cdot|s_t))$ denotes the entropy of the policy at state s_t , which measures how “random” the policy is.

This entropy-augmented objective is widely used because it helps prevent convergence to local optima and encourages exploration by rewarding policies that remain

stochastic rather than prematurely collapsing to deterministic behaviours.

Soft Actor-Critic is an approximate policy iteration algorithm. Essentially, given a policy, the algorithm estimates its soft Q-values and then updates the policy toward the best one. SAC uses neural networks to represent both the policy and the value functions, updating them iteratively and alternately without waiting for full convergence at each step.

The practical implementation introduces several challenges, which are addressed through techniques such as double Q-value clipping, target network updates, and a squashed Gaussian policy. In the implementation, the version provided by the Stable-Baselines3 library will be used.

E. Domain Randomization

As mentioned, one of the main challenges in robotic Reinforcement Learning is transferring policies from simulation to the real world, due to model or parameter mismatches. A widely used approach in recent years is domain or dynamics randomization [7], [8]. The core idea is to train the policy across a variety of simulated environments by randomly varying simulation parameters. This produces a more general policy that can perform well across different scenarios, including the real world, and reduces the risk of overfitting to a single simulated environment, a particular concern when using deep neural networks.

Formally, the objective can be written as:

$$\max_{\theta} \mathbb{E}_{\xi \sim p(\phi)} \left[\mathbb{E}_{\pi_{\theta}} \left[\sum_t \gamma^t r_t \right] \right]$$

where ϕ is the parameter that control randomization. The simplest method is **Uniform Domain Randomization (UDR)**: for each simulation parameter, an upper and lower bound are defined, and values are sampled uniformly at the start of each episode.

F. SimOpt: A dynamical System Identification

Domain randomization is not straightforward to apply. The choice of the randomization space is critical: if the space is too large, the learned policy may tends to be overly conservative; if it is too narrow, the policy may lacks generalization. Furthermore, the randomization ranges often need to be manually tuned, in a manner similar to system identification, which can require repeated real-world iterations to obtain a configuration suitable for transfer.

Several approaches have been proposed to address this challenge. Methods such as Active Domain Randomization (ADR) [6] or DORAEMON [11] rely solely on simulation, aiming to learn general policies without any access to the target environment. Such methods are

particularly useful in scenarios where direct interaction with reality is impractical, such as underwater robotics or space robotics. Other methods, such as DROPO [12] and SimOpt [13], adapt the randomization distribution by incorporating feedback from real-world rollouts.

The approach considered here is inspired by SimOpt and similar methods. It follows an iterative scheme in which a policy is trained in simulation under domain randomization, then evaluated through real-world rollouts. The simulated and real trajectories are compared, and the distribution of randomized dynamics parameters is updated to better approximate the real system. This strategy can be viewed as an intermediate approach between pure domain randomization and system identification.

Formally, let the dynamics distribution be parameterized by ϕ . The objective is to find the best randomization parameter, minimizing the discrepancy between simulated and real rollouts under the current policy π_{θ} :

$$\min_{\phi} \mathbb{E}_{\xi \sim p(\phi)} \left[\mathbb{E}_{\pi_{\theta}} \left[D(\tau_{\text{sim}}^{\xi}, \tau_{\text{real}}) \right] \right],$$

where $D(\cdot, \cdot)$ is a trajectory discrepancy metric (e.g., based on states or observations).

The SimOpt algorithm can be described as follows:

Algorithm 2 SimOpt Algorithm

- 1: Initialize dynamics parameter distribution $p(\phi)$ and policy parameters θ
 - 2: **for** iteration $k = 1, \dots, K$ **do**
 - 3: Sample dynamics parameters $\xi \sim p(\phi)$
 - 4: Train policy π_{θ} in simulation with dynamics ξ
 - 5: Collect simulated rollouts τ_{sim}^{ξ} using π_{θ}
 - 6: Collect real-world rollouts τ_{real} using π_{θ}
 - 7: Evaluate discrepancy: $D(\tau_{\text{sim}}^{\xi}, \tau_{\text{real}})$
 - 8: Update dynamics distribution $p(\phi)$ to minimize the discrepancy
 - 9: Update policy π_{θ} using the updated $p(\phi)$
 - 10: **end for**
-

In the specific implementation used here, the details of the distribution update are described in the experimental section.

III. EXPERIMENTS

A. Experimental Setup

The experiments are conducted in simulation using the Hopper environment of the MuJoCo physics engine [14]. The Hopper is a two-dimensional one-legged robot consisting of a torso, thigh, leg, and a single foot. The dynamic simulation parameters of interest are the leg's masses. The goal of the Hopper is to move forward by controlling three hinges that connect the four parts of the robot.

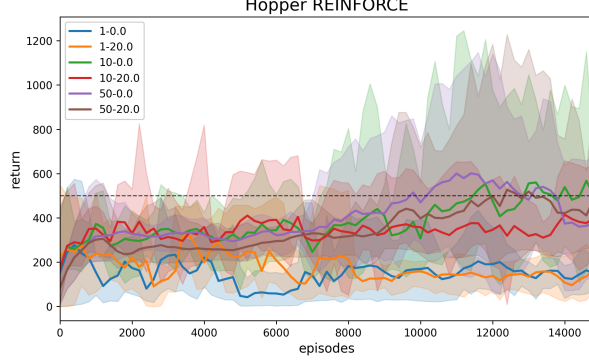


Fig. 1. Return obtained with REINFORCE

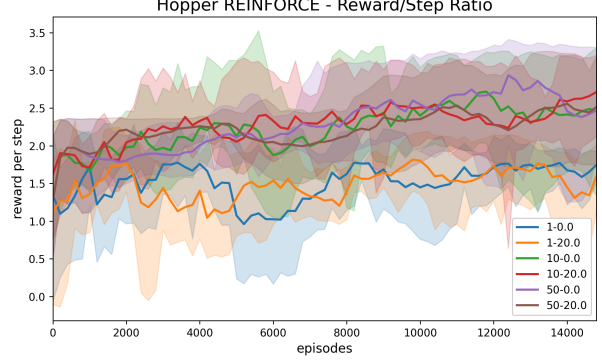


Fig. 2. Reward per step obtained with REINFORCE

The environment is characterized by a continuous state vector with 12 components: the x -coordinate of the torso, the z -coordinate of the torso, the angles of the torso, thigh, leg, and foot, which together determine the robot’s position in space, as well as the corresponding velocities for each position variable. Each component takes values in a continuous range $(-\infty, +\infty)$.

The Hopper receives as observation the full state vector except for the x -coordinate of the torso. The action consists of three components: the torque applied to the thigh rotor, the torque applied to the leg rotor, and the torque applied to the foot rotor. Each action component is continuous and spans $[-1, 1]$. The reward function is defined as:

$$\text{reward} = \text{healthy_reward} + \text{forward_reward} - \text{ctrl_cost}$$

where:

- **healthy_reward** is 1 if the Hopper is in a healthy state (i.e., its state lies within predefined bounds).
- **forward_reward** is a reward for moving forward, computed as $\frac{dx}{dt}$ where dx is the x -axis displacement and dt is the time step.
- **ctrl_cost** is a penalty that discourages extreme actions.

The episode terminates if the Hopper becomes unhealthy, and is truncated if it reaches 500 steps. Two scenarios are considered: a source scenario, which serves as the basis for all training, and a target scenario, which simulates a real-world environment. In the source scenario, the torso mass is 30% lighter than in the target scenario, representing an unmodelled effect. The goal is to achieve policy transfer from the source to the target, overcoming the inconsistent modelling.

B. Results of Classical Algorithms

Classical batch algorithms were first analysed. The implementation of REINFORCE, both with and without a baseline, follows the standard formulation. For Actor-Critic, a batch implementation [15] was adopted,

employing bootstrapped advantages (TD) and baseline advantages derived from the value function. Several implementation tricks were introduced to improve performance. Advantages were normalized at the batch level (rather than per episode), and gradient clipping was applied to stabilize training. In the Actor-Critic algorithm, the actor and critic were implemented as separate neural networks without shared features. All networks are feed-forward with the same architecture: an input layer matching the observation space, a hidden layer with 128 neurons, and an output layer of action dimension. The policy was modelled as a Gaussian distribution, with mean and variance predicted by the actor network. The optimizer used for all networks was Adam with a learning rate of $1e-3$.

For REINFORCE, the significant hyperparameters are batch size, baseline value, and discount factor, summarized in Table I. Each setup was evaluated using Optuna with pruning enabled. The evaluation metric was defined as the difference between the average return and the average number of steps across a batch of episodes. The motivation for this metric, in terms of reflecting meaningful performance improvements, is discussed later.

TABLE I
HYPERPARAMETER RANGES AND BEST VALUES FOR REINFORCE TUNING.

Hyperparameter	Values / Choices	Best
gamma	{0.95, 0.99}	0.99
baseline	{0.0, 20.0, 50.0, 100.0}	20.0
batch size	{10, 50}	10

The best-performing hyperparameters were: baseline = 20.0, batch size = 10, and discount factor = 0.99. To further investigate algorithm behavior, additional experiments were conducted with batch sizes of 1, 10, and 50 and baseline values of 0 and 20, over runs of 15,000 episodes (approximately 200,000 steps).

The results highlight several important trends (see Figures 1 and 2). Training was generally unstable, partic-

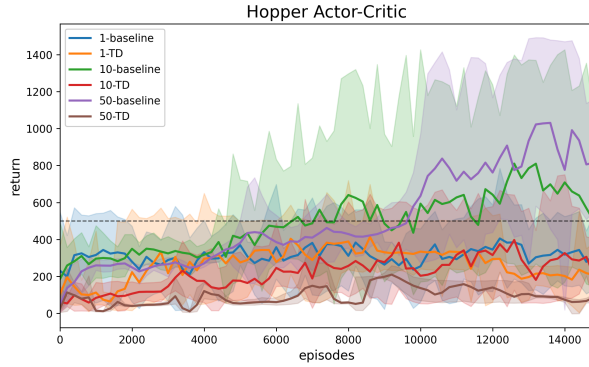


Fig. 3. Return obtained with Actor Critic

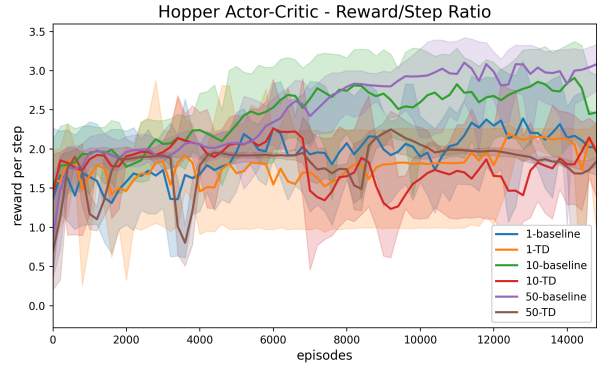


Fig. 4. Reward per step obtained with Actor Critic

ularly for single-batch runs, which exhibited significant noise. A batch size of 10 performed substantially better than 1, producing more frequent and stable updates. In shorter runs, batch size 10 outperformed batch size 50; however, in longer runs, batch size 50 achieved the best overall performance. Although the final returns remain unstable, they reach approximately 800–900 with a larger batch size. Over extended runs, the robot starts to execute physical steps, but it cannot sustain equilibrium for long durations.

The algorithm exhibited a tendency to get stuck in local minima, especially when trained with few episodes per run. In these cases, the agent often remained stationary, collecting only the healthy reward without significant movement. This behaviour suggests that alternative evaluation metrics, such as return minus steps or return per step, provide a more accurate measure of performance. Another phenomenon was observed: a local maximum in which the Hopper adopted an unstable walk with its torso leaning excessively forward, a “bull-like” posture. Although this behaviour maximized reward, it was physically unrealistic. Due to computational constraints, experiments were not extended beyond 20,000 episodes. In the limited longer runs conducted, no further improvement was observed.

For the batch Actor-Critic algorithm, the performance was better overall, especially when using the baseline (i.e., unbiased advantages), reaching returns in the range of 1000 and above. The stability of updates improved in both cases, as expected from variance theory; however, the gain was modest and did not represent a substantial leap compared to REINFORCE, as the Hopper was still unable to learn a perpetual walking gait. The corresponding results are reported in Figures 3 and 4.

C. Results of SAC

For the SAC algorithm, the implementation from the **stable-baselines3** library [16] was employed. Several hyperparameters required tuning, including the

batch size for updates, the discount factor, the soft update coefficient, and the network architecture. As in the classical algorithms, hyperparameter selection was performed using Optuna at the beginning of the pipeline. The search was conducted on the source environment, and the selected values were subsequently applied to all environments. Table II summarizes the hyperparameter ranges and chosen values. Training was carried out for 200,000 steps. No buffer size limitation was imposed, and 5,000 steps were collected before the start of training. Updates were performed at every step with a single gradient update. The entropy coefficient (*ent_coef*) was set to be automatically adjusted during training.

TABLE II
HYPERPARAMETER RANGES AND BEST VALUES FOR SAC TUNING.

Hyperparameter	Range / Values	Best
learning rate	[1e-5, 3e-3]	3e-4
tau	[0.005, 0.02]	0.01
gamma	[0.95, 0.995]	0.995
batch size	{64, 128, 256, 512}	64
network architecture	{64, 128, 256}	128

The SAC algorithm represents a significant improvement compared to the classical methods. Final performance reached an average return of approximately 1500, with a reward per step close to 3. Under these conditions, the Hopper was consistently able to walk without difficulty, and episodes typically ended by truncation rather than termination.

Training exhibited some instability in the early stages. Log analysis and video inspection indicated that the agent occasionally converged to local minima (e.g., remaining stationary). However, the entropy regularization term appeared to facilitate exploration and ultimately led to improved policies. The algorithm also demonstrated stability with respect to different random seeds, consistently achieving similar performance. Detailed results are provided in Table III.

D. Results of Domain Randomization and SimOpt

The transfer performance from the source to the target environment was observed to be suboptimal. While the source environment achieved a return in the range of 1500, representing a stable walking motion that could, in theory, continue indefinitely, the target environment performance dropped to approximately 1000–1200 when using the source policy, corresponding to a walking behaviour that becomes unstable toward the end.

Uniform Domain Randomization (UDR) was applied by scaling each mass independently within the range $[0.5, 1.5]$ with uniform probability. The technique demonstrated increased instability during training and across seeds (see Table III), and may have only marginally improved the lower bound of transfer performance. In contrast, SimOpt appeared to be more effective under specific conditions.

For SimOpt, rather than computing the discrepancy from l_1 or l_2 norms of Gaussian smoothed trajectories as in the original work [13], a transition-based discrepancy measure was preferred:

$$\mathbb{E}_{(s,a,s') \sim \mathcal{D}} [\|f_{\xi}(s,a) - s'\|^2]$$

where (s, a, s') represents a transition from state s under action a , the dataset $\mathcal{D}$ is collected from the real system under the chosen policy, and $f_{\xi}(s, a)$ is the simulator’s transition function for dynamics parameter ξ , which is deterministic in the Hopper case.

This transition-based approach offers several advantages. Unlike trajectory-level measures, it does not accumulate errors from previous steps, allowing the simulator to be used directly for discrepancy computation. Experimental results indicate that this measure is more informative than average trajectory discrepancy, providing a clearer distinction between different simulation settings. The discrepancy is computed using the full state rather than observations, i.e., including the x-axis of the torso, which is fully accessible to the human programmer and therefore does not constitute a limitation in this context.

Not all trajectories are equally informative for estimating a good dynamics parameter distribution. Short or low-activity trajectories provide limited information compared to longer trajectories that encompass diverse and crucial movements. Therefore, initiating the iterative update phase prematurely can reduce the effectiveness of the algorithm. The training pipeline begins by learning a sufficiently effective policy in the source or UDR environment. Subsequently, Relative Entropy Policy Search (REPS) [17] is applied: starting from a prior multivariate normal distribution, multiple parameter vectors are sampled and their discrepancies are computed. These discrepancies are used in an exponential weighting scheme to update the mean and covariance of the distribution, which then serves as the new prior.

This process is repeated for several iterations. The best scaling factors found, used for the SimOpt training, are $[1.114, 1.089, 0.980]$ for the three variable masses.

Empirically, REPS has been observed to be more robust than black-box optimizers such as CMA-ES or Particle Swarm Optimization, likely because it considers multiple effects simultaneously and avoids overfitting to marginally better parameter sets. Finally, a complete training phase is executed using the updated distribution centred at the new mean. Although SimOpt has demonstrated promising results, its value primarily lies in the general methodology rather than the specific implementation. Different environments present distinct dynamics, trajectory characteristics, and discrepancy metrics, making the selection of the discrepancy function and the scheduling of updates highly task-dependent. Consequently, careful adaptation to the specifics of each application is required.

IV. CONCLUSION

During this work, an introduction to Reinforcement Learning and the Sim2Real problem was provided. Classical algorithms were analysed, highlighting both their simplicity and their limitations, such as training instability and a tendency to converge to local minima. Modern algorithms, such as SAC, were then examined, demonstrating significantly faster and more stable learning, although occasional unexpected behaviours were observed during early training. Domain Randomization was subsequently applied to address the sim-to-real transfer problem. Among the methods considered, SimOpt showed strong potential, achieving competitive performance on the target environment when paired with an appropriate entropy maximization algorithm for distribution update.

Several limitations should be acknowledged. Due to computational constraints, a thorough evaluation across all scenarios and seeds was not feasible, and the hyperparameter search space was necessarily restricted. Furthermore, the SimOpt implementation is sensitive to the scheduling of updates and the choice of discrepancy metric, making its generalization to other environments non-trivial.

Future work could focus on a more systematic analysis of dynamics identification methods, a broader comparison of randomization strategies, and the development of approaches that improve robustness without incurring overfitting to the source environment.

REFERENCES

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “Playing Atari with Deep Reinforcement Learning,” arXiv preprint arXiv:1312.5602, 2013.

TABLE III
MEAN RETURN ($\pm$ STD) EVALUATED ON EACH TEST ENVIRONMENT, FOR EACH TRAINING CONFIGURATION.

Test Environment	Training Environment							
	Source		Target		UDR		SimOpt	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
Source	1503.43	17.27	/	/	1219.51	333.87	1542.59	2.45
Target	1297.16	28.85	1526.25	13.05	820.61	372.60	1595.06	6.25
UDR	/	/	/	/	1157.35	365.50	/	/
SimOpt	/	/	/	/	1062.58	406.98	1549.65	108.86

- [2] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis, “Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm,” *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
- [3] J. Achiam, “Spinning Up in Deep Reinforcement Learning,” OpenAI, 2018.
- [4] P. Kormushev, S. Calinon, and D. G. Caldwell, “Reinforcement Learning in Robotics: Applications and Real-World Challenges,” *Robotics*, vol. 2, no. 3, pp. 122–148, 2013. :contentReference[oaicite:0]index=0
- [5] J. Kober, J. A. Bagnell, and J. Peters, “Reinforcement Learning in Robotics: A Survey,” *Int. J. Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [6] OpenAI, I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang, “Solving Rubik’s Cube with a Robot Hand,” *arXiv preprint arXiv:1910.07113*, 2019.
- [7] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30, 2017.
- [8] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-Real Transfer of Robotic Control with Dynamics Randomization,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3803–3810, 2018.
- [9] L. Döring, “The Mathematics of Reinforcement Learning,” Available: https://www.wim.uni-mannheim.de/media/Lehrstuehle/wim/doering/RL/RL_VORLESUNG.pdf, 2024.
- [10] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,” Available: <https://arxiv.org/abs/1801.01290>, 2018.
- [11] G. Tiboni, P. Klink, J. Peters, T. Tommasi, C. D’Eramo, and G. Chalvatzaki, “Domain Randomization via Entropy Maximization (DORAEMON),” Available: <https://arxiv.org/abs/2311.01885>, 2023.
- [12] G. Tiboni, K. Arndt, and V. Kyriki, “DROPO: Sim-to-Real Transfer with Offline Domain Randomization,” *Robotics and Autonomous Systems*, vol. 166, p. 104432, 2023. Available: <https://doi.org/10.1016/j.robot.2023.104432>.
- [13] Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox, “Closing the Sim-to-Real Loop: Adapting Simulation Randomization with Real World Experience,” *arXiv preprint arXiv:1810.05687*, 2018. Available: <https://arxiv.org/abs/1810.05687>
- [14] Gymnasium, “Hopper Environment,” Available: <https://gymnasium.farama.org/environments/mujoco/hopper/>.
- [15] S. Levine, “Actor-Critic Algorithms,” *CS 285: Deep Reinforcement Learning*, University of California, Berkeley, 2019. Available: <https://rail.eecs.berkeley.edu/deeprlcourse/static/slides/lec-6.pdf>.
- [16] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and J. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” Available: <https://github.com/DLR-RM/stable-baselines3>, 2019. Accessed: Sep. 11, 2025.
- [17] K. Cui and M. Hensel, “Likelihood-free Inference in Reinforcement Learning: Relation to REPS and Sim-to-Real Applications,” Available: <https://www.ias.informatik.tu-darmstadt.de/uploads/Team/BorisBelousov/likelihood-free-inference.pdf>, 2020. Accessed: Sep. 11, 2025.